Load Balancer Setup

with Nginx on AlmaLinux

High-Availability Web Infrastructure | Step-by-Step Guide

April 2026 | AlmaLinux 10 | Nginx 1.26 | PHP-FPM

Group-2

1. Suon Pisey
2. Nem Sothea
3. Sourn Savourn
4. Oun Sreynich
5. Moeun Nithvaraman

OVERVIEW

Table of Contents

01 Project Overview

02 Why Use a Load Balancer?

03 Architecture & Process Flow

04 Environment Details

05 Phase 1 : VM1 - Websites + Dashboard

06 Phase 1 : Configuration & SELinux

07 Phase 2 : VM2 - Load Balancer Setup

08 Demo : Site A & Site B

09 Demo : Health Dashboard

10 Demo : Nginx Load Balancer in Action

11 Verification & Testing

12 Challenges & Solutions

13 Extending the Infrastructure

14 Conclusion

Project Overview

Two virtual machines working together as a high-availability web infrastructure

VM1 - Web Server (192.168.1.8)

- Site A on port 8081 (Primary Web App)
- Site B on port 8082 (Secondary Web App)
- Health Dashboard on port 8080
- PHP-FPM for dynamic content
- Real-time uptime monitoring

VM2 - Load Balancer (192.168.1.20)

- Nginx reverse proxy on port 80
- Round-robin traffic distribution
- Proxies to VM1:8081 and VM1:8082
- Automatic failover on backend failure
- Single public entry point for users

Why Use a Load Balancer?

High Availability

If one backend fails, traffic is automatically redirected to the healthy one.

Scalability

Add more backend servers without changing the public endpoint.

Traffic Distribution

Prevents overload on a single server using round-robin, least connections, etc.

Zero-Downtime Maintenance

Take one backend offline while the other continues serving requests.

Simplified SSL/TLS

Terminate certificates on the load balancer only - backends stay simple.

Architecture & Process Flow

Health Dashboard

Port 8080

Auto-refresh every 10s

Monitors both backends

Response time in ms

Client Request

VM2 - Nginx Load Balancer (port 80)

upstream: 192.168.1.8:8081 | 192.168.1.8:8082

Algorithm: Round-Robin | Failover: Automatic

Site A (port 8081)

Primary Web Application

VM1: 192.168.1.8

Site B (port 8082)

Secondary Web Application

VM1: 192.168.1.8

Request Flow

1. User accesses VM2 IP
2. Nginx selects backend
3. Proxied to :8081/:8082
4. Backend responds
5. Dashboard health check

Environment Details

Operating System

AlmaLinux 10 (RHEL-compatible)

Web Server

Nginx 1.26

Dynamic Content

PHP-FPM 8.x

Firewall

firewalld (optional)

SELinux

Enforcing (custom booleans)

VM1 IP Address

192.168.1.8

VM2 IP Address

192.168.1.20

Dashboard Port

8080 (health monitoring)

Phase 1: VM1 - Two Websites + Dashboard

Step 1: Install Nginx & PHP-FPM

sudo dnf install -y nginx php php-fpm

sudo systemctl enable --now nginx php-fpm

Provides web serving + dynamic PHP content

Step 2: Create Website Directories

/var/www/site_a/html (Site A)

/var/www/site_b/html (Site B)

/var/www/dashboard/html (Dashboard)

Step 3: Create Sample Pages

index.html with welcome message

uptime.php for live server uptime

Each site has unique styling & identity

Step 4: Configure Nginx Virtual Hosts

site_a.conf - listen 8081

site_b.conf - listen 8082

dashboard.conf - listen 8080 (PHP)

Phase 1: Configuration & SELinux

Nginx Virtual Host Configuration (site_a.conf)

```
server {
    listen 8081;
    root /var/www/site_a/html;
    index index.html index.php;

    location ~ \.php$ {
        fastcgi_pass unix:/run/php-fpm/www.sock;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME
            $document_root$fastcgi_script_name;
    }
}
```

SELinux Configuration

```
# Allow custom ports
sudo semanage port -a -t http_port_t -p tcp 8081
sudo semanage port -a -t http_port_t -p tcp 8082
sudo semanage port -a -t http_port_t -p tcp 8080

# Allow network connections
sudo setsebool -P httpd_can_network_connect 1
```

Firewall Rules

```
sudo firewall-cmd --permanent --add-port={8081,8082,8080}/tcp
sudo firewall-cmd --reload
```

Validate & Restart

```
sudo nginx -t && sudo systemctl restart nginx php-fpm
```

Phase 2: VM2 - Nginx Load Balancer

Load Balancer Config (/etc/nginx/conf.d/load_balancer.conf)

```
upstream backend_servers {
    server 192.168.1.8:8081;
    server 192.168.1.8:8082;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /dashboard {
        proxy_pass http://192.168.1.8:8080;
    }
}
```

Step 1: Install Nginx

```
sudo dnf install -y nginx
```

Step 2: SELinux Boolean

```
sudo setsebool -P httpd_can_network_connect 1
```

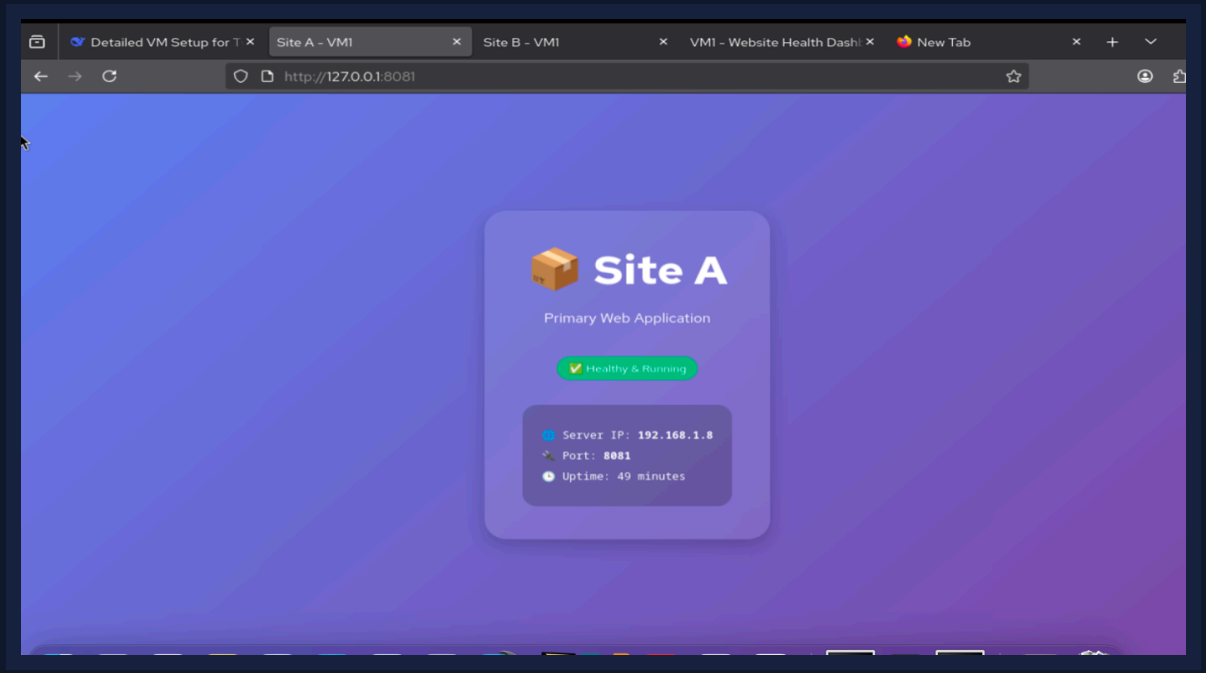
Step 3: Open Firewall

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

Step 4: Start Nginx

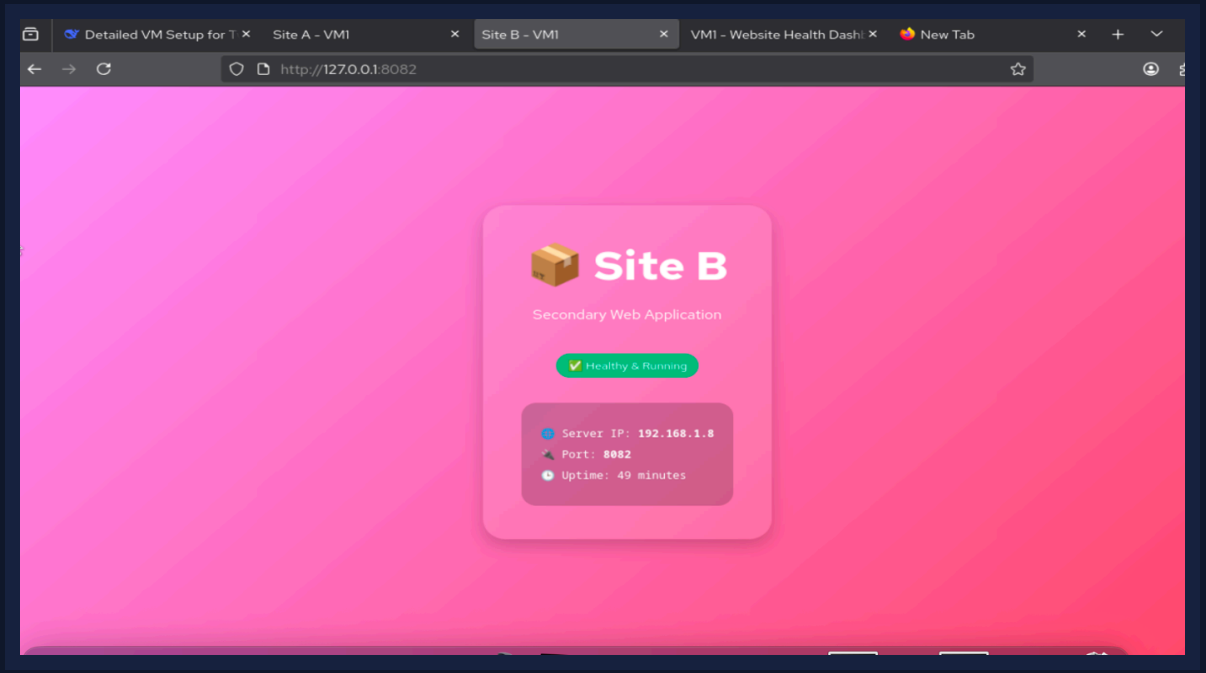
```
sudo systemctl enable --now nginx
```

Demo: Site A & Site B



Site A - Primary Web Application (port 8081)

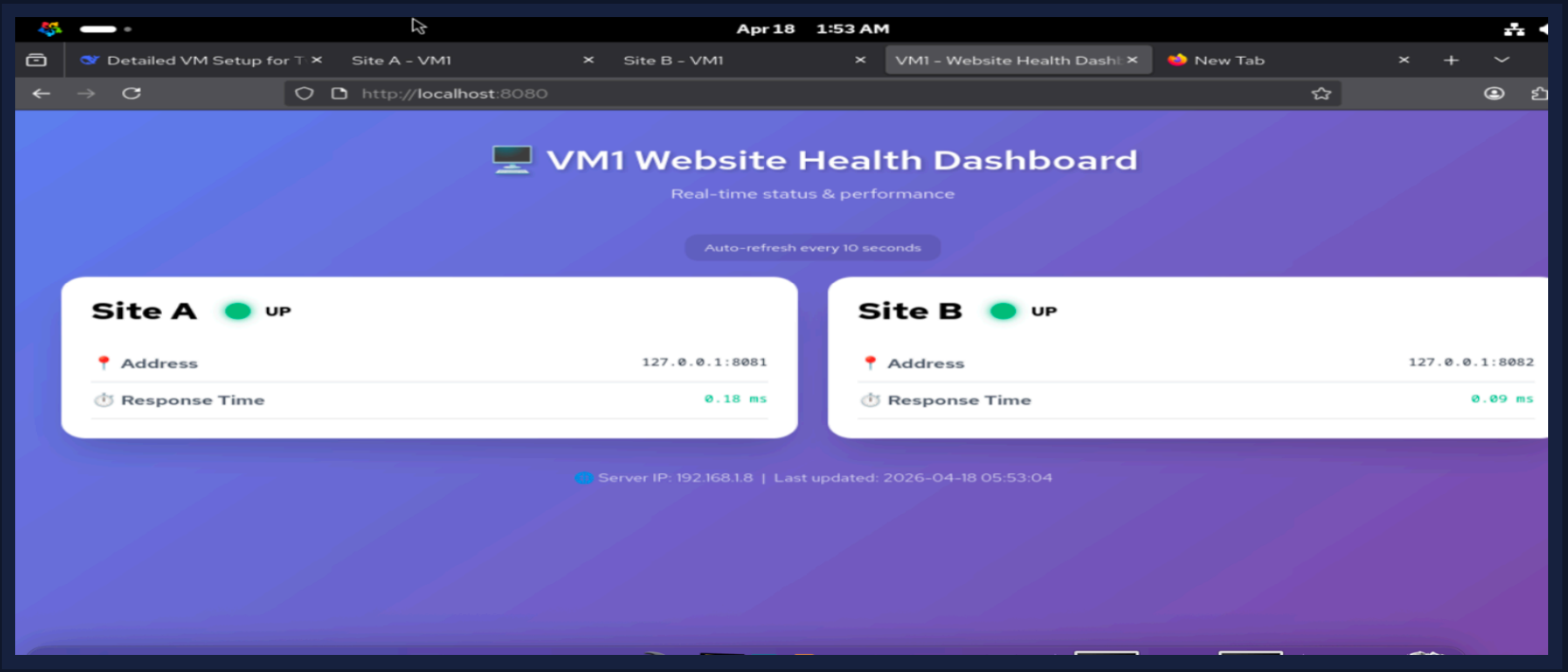
Server IP: 192.168.1.8 | Healthy & Running | Uptime: 49 min



Site B - Secondary Web Application (port 8082)

Server IP: 192.168.1.8 | Healthy & Running | Uptime: 49 min

Demo: Health Dashboard



Auto-Refresh

Updates every 10 seconds automatically

Response Time

Measures latency in milliseconds per backend

Status Monitor

Green = UP, Red = DOWN
Real-time health check

Dual Backend

Monitors Site A (:8081) and Site B (:8082)

Demo: Nginx Load Balancer in Action

```
MV2 [Running]
Apr 18  1:56 AM
vm2@localhost:~
sudo nginx -t && sudo systemctl reload nginx
nginx: [emerg] invalid value "0" in "keepalive" directive in /etc/nginx/conf.d/load_balancer.conf:2
nginx: configuration file /etc/nginx/nginx.conf test failed
vm2@localhost:~$ curl http://localhost
curl http://localhost
curl http://localhost
<h1>Welcome to Site A (Port 8081)</h1>
<h1>Welcome to Site B (Port 8082)</h1>
<h1>Welcome to Site A (Port 8081)</h1>
vm2@localhost:~$ # On VM2, tail the log
sudo tail -f /var/log/nginx/access.log
::1 - - [18/Apr/2026:00:52:27 -0400] "GET / HTTP/1.1" 502 157 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:00:52:27 -0400] "GET / HTTP/1.1" 502 157 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:00:52:27 -0400] "GET / HTTP/1.1" 502 157 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:00:53:56 -0400] "GET / HTTP/1.1" 502 157 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:03:08 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:03:08 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:03:08 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:04:30 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:04:30 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
::1 - - [18/Apr/2026:01:04:30 -0400] "GET / HTTP/1.1" 200 39 "-" "curl/8.12.1" "-"
^C
vm2@localhost:~$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabl>
     Active: active (running) since Sat 2026-04-18 00:54:45 EDT; >
 Invocation: 208828772c0b445a8bcb3adbc115d01a
    Process: 1204 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (cod>
    Process: 1228 ExecStartPre=/usr/sbin/nginx -t (code=exited, s>
    Process: 1267 ExecStart=/usr/sbin/nginx (code=exited, status=>
    Process: 3724 ExecReload=/usr/sbin/nginx -s reload (code=exit>
   Main PID: 1277 (nginx)
      Tasks: 3 (limit: 10541)
     Memory: 2.7M (peak: 4.8M, swap: 1.2M, swap peak: 2.6M)
        CPU: 36ms
     CGroup: /system.slice/nginx.service
             ├─1277 "nginx: master process /usr/sbin/nginx"
```

Round-Robin Proof

curl shows alternating
Site A / Site B responses

Access Logs

tail -f shows distributed
requests across backends

Service Status

systemctl shows nginx
active (running) on VM2

HTTP 200 OK

All responses return
200 - backends healthy

Verification & Testing

Site A Direct

```
curl http://192.168.1.8:8081
```

Expected: Welcome message or styled page

Site B Direct

```
curl http://192.168.1.8:8082
```

Expected: Welcome message or styled page

Dashboard

```
curl http://192.168.1.8:8080
```

Expected: Health status JSON or HTML

Load Balancer

```
curl http://192.168.1.20 (x3)
```

Expected: Alternating: Site A, B, A...

Dashboard via LB

```
curl http://192.168.1.20/dashboard
```

Expected: Dashboard HTML page

Failure Test

```
sudo systemctl stop nginx (VM1)
```

Expected: VM2 stops routing to failed backend

Challenges & Solutions

CHALLENGE

SELinux Blocking Ports

SOLUTION

Used semanage port -a to add custom HTTP ports

CHALLENGE

PHP-FPM Network Error

SOLUTION

Enabled httpd_can_network_connect SELinux boolean

CHALLENGE

Port 80 Conflict

SOLUTION

Stopped & removed HAProxy that was occupying the port

CHALLENGE

Nginx Syntax Errors

SOLUTION

Replaced config & validated with nginx -t before restart

CHALLENGE

Emoji Display Issues

SOLUTION

Added <meta charset="UTF-8"> to all HTML pages

CHALLENGE

PHP Socket Permission

SOLUTION

Fixed via SELinux boolean + PHP-FPM socket permissions

Extending the Infrastructure

Add More Backends

Add new server lines in upstream block.
No client-side changes needed -
instant horizontal scaling.

LB Algorithms

Switch from round-robin to:
least_conn; ip_hash; random;
for different traffic patterns.

Enable SSL/TLS

Obtain certificate (Let's Encrypt)
and add listen 443 ssl; on VM2.
Backends use plain HTTP.

Session Persistence

Use sticky cookie (Nginx Plus)
or ip_hash for session affinity.
Keeps user on same backend.

Logging & Monitoring

Access logs show which upstream
handled each request. Integrate
Grafana or Prometheus.

Conclusion

- Production-ready, scalable, and monitored web infrastructure
- Built entirely with open-source tools (Nginx + PHP-FPM)
- Load balancer ensures high availability with automatic failover
- Health dashboard provides real-time visibility into backend status
- SELinux enforcing mode maintained for enterprise-grade security
- All common challenges addressed with practical solutions

Group-2

1. Suon Pisey
2. Nem Sothea
3. Sourn Savourn
4. Oun Sreynich
5. Moeun Nithvaraman

AlmaLinux 10 | Nginx 1.26 | PHP-FPM 8.x | April 2026

Thank You!